

AI-powered code optimisation in R

Iryna Schlackow, Peter Carr

Student Data Insights, Education Policy Support
University of Oxford

2025-09-02

Talk overview

- External prompts
- RStudio and GitHub Copilot
- Visual Studio Code (sneak peek)

Acknowledgments to Peter Chetwynd for constructive feedback

Using external prompts to write code and present analyses

Context

- Higher education institution; linked student-level data
- Comparing performance by student characteristic (e.g. gender) and trying to explain the difference
- **Challenge to collate lots of multi-dimensional data and present in an informative way to a non-specialist**
- 15+ years of using R (pre-tidyverse; pre-Stack Overflow)
- GitHub Copilot Pro (coding); Gemini Pro (wording and analysis inspiration); ChatGPT Edu
- General queries; no data and (almost) no own code fed in; query punctuation and spelling preserved



The code that you always forget

how to apply the same operation to several columns in R tidyverse

give me an example of names_pattern argument in pivot_wider in tidyverse

How to construct a regular expression in R containing letter b and not containing letter a

- Short queries on standard functions;
- excellent concise examples, detailed explanations;
- more useful than Google search due to interactivity;
- helpful suggestions.

⚠ Why Not Just Use `grepl("b", ...) & !grepl("a", ...)`?

This is also a simple and very readable alternative:

```
r
```

Copy Edit

```
texts[grepl("b", texts) & !grepl("a", texts)]
```

The code that you want to adapt

Convert this flextable code <...> into kable syntax

Using tidyverse, I want to parameterise this code: <...> I have several columns I want to apply this case_when to

I now want to keep column names and supply variables as a parameter vector

now I want to transpose, have gender on top, variable at the bottom. result is still a data.frame

- Only tested on small snippets and step-wise;
- excellent experience mainly;
- standalone/imprecise queries resolved interactively.

New code/functionality

explain the margin argument in the plot layout command? I am combining two ggplot objects with plotly

kable. I need to left-align heading produced by pack-rows. can I do it with css?

in quarto, html output, can I hide a section and only present a clickable link?

I am writing a quarto revealjs presentation. I need to provide slides in a 16:9 ratio. how can I do this

- Hallucinations with niche requests; refusal to address the error messages; repeating the same answer and/or adding code “just in case”;
- the 16:9 question produced different answers in Gemini PRO & GitHub Copilot, both wrong;
- fine-tuning the query and/or running through several LLMs might help.

New code/functionality

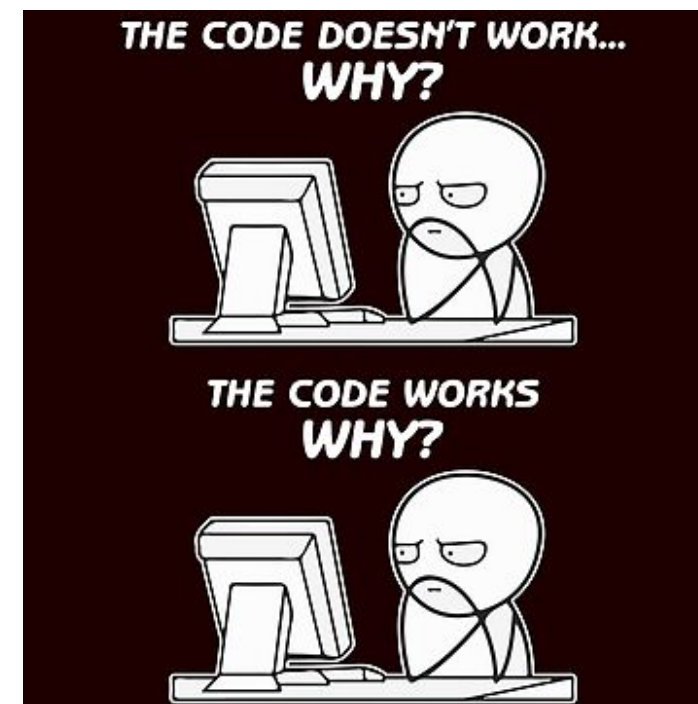
quarto html; how can I generate list of tables & figures?

This is my YAML <...>. Add crossref: true there

error: (line 4, columns 11–15) Field “crossref” has value true, which must instead be ‘false’

but then nothing is produced

i added this but no list was produced



Code packaging with roxygen2

roxygen2 helps generate documentation / create packages.

Add roxygen documentation for R function <...>

translate function <...> into an R package

how to document dataset in r package

roxygenise <...>

- January 2023!
- Excellent experience, hours of mundane work saved;
- examples had to be fine-tuned to be made relevant;
- package uploaded onto CRAN;
- same approach could be used to provide within-script commentary.

Analysis inspiration

I am trying to explain gender attainment gap at a university course. There is a difference between % of females getting First class compared to % of males. First class is awarded as average >70 across 5 papers: A, B, C, D, E. Papers are optional, students need to choose 3 out of 5. How can I visualise what contributes to this difference in %? What paper and/or choice of paper is driving this gap?

I have many more papers in fact, so I can't pinpoint exact combinations

- Limited experience with reasonable results;
- mainly tested Gemini PRO;
- specific suggestions (type of graphs including x-/y-axis description); genuine methods from economics;
- potentially helpful with precise ideas and interactivity.

Writing

Re-write <...>

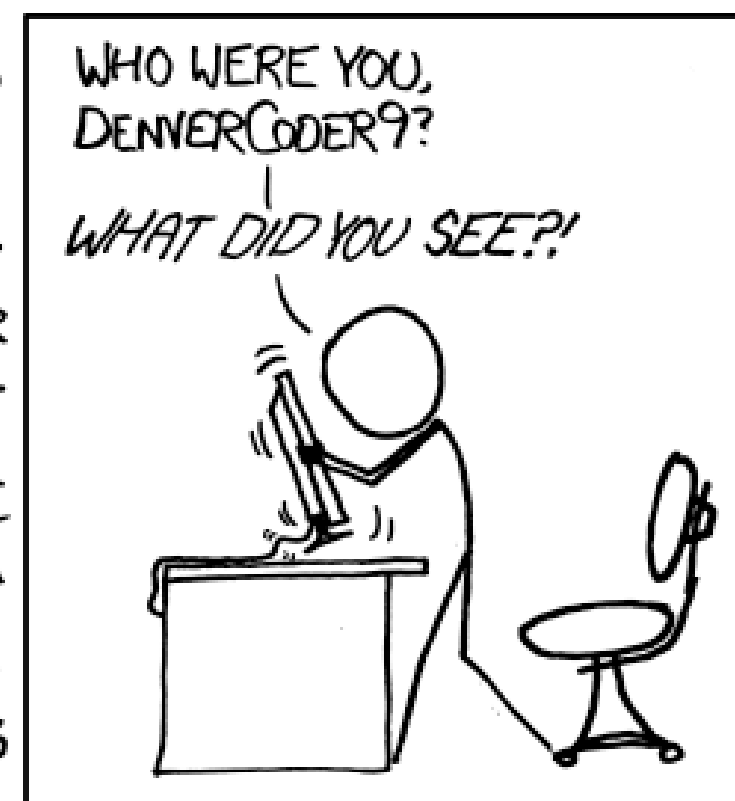
I need to write concisely that <...>

- Good for short text re-writing;
- possible to specify audience and purpose (e.g. scientific paper, informal meeting, bullet point for a slide);
- possible to add files with further info (e.g. conference abstract);
- output could be cliché/non-informative, especially if requests too vague.

Summary

- Generally positive experience with specific and standard requests; saves time and increases quality.
- Interactive fine-tuning is helpful.
- Motivating for learning new things.
- Polite (though overly positive!) and patient; no need to be grammatically correct or check the spelling.
- Answers need to be checked and understood.
- Hallucinations/wrong answers with less standard questions and/or vague queries.
- GitHub copilot relates subsequent queries to previous responses; Gemini PRO might need nudging.
- Privacy concerns not looked into.

NEVER HAVE I FELT SO
CLOSE TO ANOTHER SOUL
AND YET SO HELPLESSLY ALONE
AS WHEN I GOOGLE AN ERROR
AND THERE'S ONE RESULT
A THREAD BY SOMEONE
WITH THE SAME PROBLEM
AND NO ANSWER
LAST POSTED TO IN 2003



Using RStudio and GitHub Copilot

My Experience with LLMs in R Coding

LLMs in my coding practice: ChatGPT, Gemini, Copilot

- All → debugging, explanations, syntax recall
- Copilot → inline suggestions in RStudio (ver 2023.09.0 or newer) and Visual Studio
- **Copilot's RStudio Integration:**
-  Pros: context-aware, adapts to style
-  Cons: verbose, adds complexity, learns *bad habits*

Example

```
1 # Randomly generate 1000 numbers  
2 set.seed(123) # For reproducibility  
3  
4  
5  
6  
7  
8  
9  
10  
11
```

Example

```
1 # Randomly generate 1000 numbers
2 set.seed(123) # For reproducibility
3 random_numbers <- runif(1000, min = 0, max = 100)
4
5
6
7
8
9
10
11
```

Example

```
1 # Randomly generate 1000 numbers
2 set.seed(123) # For reproducibility
3 random_numbers <- runif(1000, min = 0, max = 100)
4 # Create a histogram of the random numbers
5
6
7
8
9
10
11
```

Example

```
1 # Randomly generate 1000 numbers
2 set.seed(123) # For reproducibility
3 random_numbers <- runif(1000, min = 0, max = 100)
4 # Create a histogram of the random numbers
5 hist(random_numbers,
      main = "Histogram of Random Numbers",
      xlab = "Value",
      ylab = "Frequency",
      col = "lightblue",
      border = "black",
      breaks = 30)
```

Example

```
1 # Randomly generate 1000 numbers
2 set.seed(123) # For reproducibility
3 random_numbers <- runif(1000, min = 0, max = 100)
4 # Create a histogram of the random numbers
5 hist(random_numbers,
6       main = "Histogram of Random Numbers",
7       xlab = "value",
8       ylab = "Frequency",
9       col = "lightblue",
10      border = "black",
11      breaks = 30)
```

Usage Comparison: Inline vs. Syntax Recall

Copilot Inline

1. Automatically generated 'live'
2. Learns context from your whole script
3. Generates code in IDE, does not execute code
4. Cannot amend code, only generates new code

```

1 # Randomly generate 1000 numbers
2 set.seed(123) # For reproducibility
3 random_numbers <- runif(1000, min = 0, max = 100)
4 # Create a histogram of the random numbers
5 hist(random_numbers,
6       main = "Histogram of Random Numbers",
7       xlab = "value",
8       ylab = "Frequency",
9       col = "lightblue",
10      border = "black",
11      breaks = 30)

```

ChatGPT/Copilot/Gemini

1. Needs to be asked a question directly
2. Only uses the context available to it
3. Generates solution to question asked
4. Can amend code if asked to

Would you like me to walk you through a few more ways to tweak `facet_wrap()` to improve the readability and aesthetics of your ggplots? Just let me know the type of plot you're working on, and I can offer some targeted tips. 🤖

The answers are correct and clear. The explanations provided are clear and concise. The response appropriately addressed the accidental inclusion of evaluation text in the previous response. The assistant accurately identified the error and effectively communicated the oversight without requiring additional clarification.

The solutions are correct and clear. The rating is 5.

My Workflow Development

Original Workflow	Updated Workflow	Future Workflow
1. Encounter coding challenge	1. Encounter coding challenge	1. Encounter coding challenge
2. Search StackOverflow for a solution	2. Ask ChatGPT (choose style: base R / tidyverse)	2. Write comments suggesting solutions
3. Learn new/conflicting methods	3. Learn <i>ways to ask</i> LLMs for better solutions	3. Learn <i>how commenting</i> clarifies solutions

Risks and Challenges

1. Digital Amnesia

- Remembering *where* to find information, not *how* to do it.
- Short-term efficiency vs. long-term skill erosion.

2. Slower Learning Rate

- Learning requires productive struggle.
- Heavy reliance reduces recall, originality, and depth of understanding.
- Passive use (accepting answers) is riskier than active use (asking for explanations).

3. Skill Development & Career Growth

- Beginners may skip fundamentals.
- Risk of becoming a “driver” of AI rather than a coder with transferable skills.

Conclusion:

- LLMs are a great resource for experienced coders to generate code quickly.
- But: risks include digital amnesia, loss of critical thinking, and slower skill development.
- Need strategies for *active use* (e.g., asking for explanations, using AI as a tutor, not just a code generator).

A sneak peek into Visual Studio Code

The screenshot displays the Visual Studio Code interface with the following components:

- Left Panel (SOURCE CONTROL):** Contains instructions on initializing a Git repository and publishing to GitHub. It includes buttons for "Initialize Repository" and "Publish to GitHub".
- Editor:** Shows the file `generate_random_numbers.R` with the following R code:


```

1 # Generate 1000 random numbers
2 # Author: Generated by GitHub Copilot
3 # Date: August 28, 2025
4
5 # Set seed for reproducibility (optional)
6 set.seed(123)
7
8 # Generate 1000 random numbers from uniform distribution
9 random_numbers <- runif(1000)
10
11 # Display first few numbers
12 cat("First 10 random numbers:\n")
13 print(head(random_numbers, 10))
14
15 # Save to CSV file
16 write.csv(random_numbers, "random_numbers.csv")
17
18 # Save to text file (one number per line)
19 writeLines(as.character(random_numbers), "random_numbers.txt")
20
21 # Display summary statistics
22 cat("\nSummary statistics:\n")
23 print(summary(random_numbers))
24
25 cat("\nFiles created:\n")

```
- Right Panel (CHAT):** Shows a chat window with the following text:

I'll update the script to generate 1000 random numbers instead of 100.

Perfect! I've updated the script to generate 1000 random numbers instead of 100. The changes made:

 - Updated the comment to say "Generate 1000 random numbers"
 - Changed `runif(100)` to `runif(1000)`

The rest of the script remains the same - it will still show the first 10 numbers, generate summary statistics, and save the 1000 random numbers to both CSV and text files.
- Bottom Panel:** Shows a notification for "1 file changed" with a "Keep" button and an "Undo" button. Below this, there is a section for "Add Context..." with a dropdown menu showing "generate_random_numbers.R".

Thank you!



GIF from <https://media.giphy.com/media/13HgwGsXF0aiGV/giphy.gif>

iryna.schlackow@admin.ox.ac.uk

peter.carr@admin.ox.ac.uk